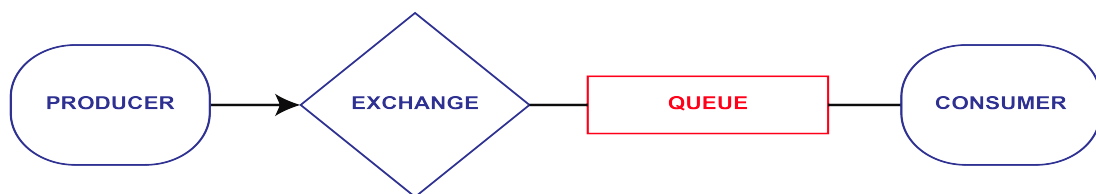




1. Introduction to RabbitMQ

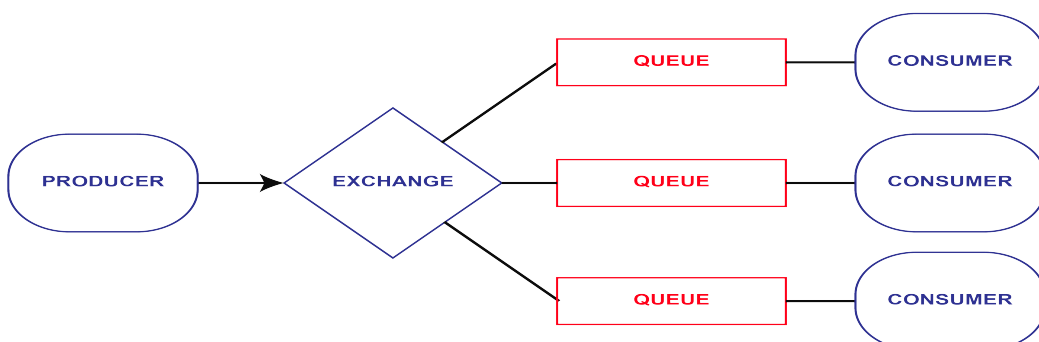
- RabbitMQ is the most widely deployed open source message broker.
- RabbitMQ is a message broker that implements AMQP (**Advanced Message Queing Protocol**).
- AMQP standardizes Messaging using Producers, Broker and Consumer.
- RabbitMQ increases loose coupling and scalability.

- Simple Systems



- Producer publishes a message to the exchange with a routing key.
- Exchange connects to the Queue with binding key and then the message will be distributed to the Queue.
- Consumer consumes the message from the queue

- Complex Systems



- Producer publishes a message to the exchange with a routing key.
- Exchange connects to the Queue with binding key and then the messages will be distributed to all or matching queues.
- Consumer consumes the message from the queue



- Message distribution from Exchange to Queue is depending on type of Exchange.
- There are different types of exchanges.

- 1) Direct Exchange
- 2) Fanout Exchange
- 3) Topic Exchange
- 4) Headers Exchange

1) Direct Exchange:

=> It routes messages to a queue by matching routing key equal to binding key.

2) Fanout Exchange

=> It ignores the routing key and sends message to all the available queues.

3) Topic Exchange

=> It routes messages to multiple queues by a partial matching of a routing key.

=> It uses patterns to match the routing and binding key.

4) Headers Exchange

=> It uses message header instead of routing key to match the queue.

2. Setup your System

1) Erlang Installation:

- a) Download Erlang from <https://www.erlang.org/downloads>

OTP 24.0 Windows 64-bit Binary File

- b) Click on the Installer
- c) Click on Next button
- d) Provide Installation Location - **E:\erl-24.0** and click on Next
- e) Click on Install button

2) Rabbit MQ Installation:

- a) Download Rabbit MQ from <https://www.rabbitmq.com/install-windows.html>

rabbitmq-server-3.9.5.exe

- b) Click on the Installer
- c) Click on Next button
- d) Provide Installation Location - **E:\MyRabbitMQ** and click on Install
- e) Click on Next button
- f) Click on Finish button



- 3) Once the installation is done, you can observe the installation directory in **E:\MyRabbitMQ**
- 4) Go to sbin directory (**E:\MyRabbitMQ\rabbitmq_server-3.7.17\sbin**) and run the following command to enable the management console on Windows

.....\sbin>**rabbitmq-plugins.bat enable rabbitmq_management**
- 5) Open <http://localhost:15672> in the browser. You can see the Rabbit MQ management console
- 6) Start the RabbitMQ Server with the following command

.....\sbin>**rabbitmq-server.bat start**
- 7) Below are some of the useful RabbitMQ commands.

rabbitmq-server.bat start
rabbitmq-server.bat stop

rabbitmq-service.bat start
rabbitmq-service.bat stop

3. DirectExchange –Example

A)Develop the Hello MicroService-1

- 1) Copy **hello-micro-service** from previous module as **hello-micro-service-1**
- 2) Update pom.xml as follows.
- 3) Update application.properties
- 4) Write HelloService.java
- 5) Write HelloController.java
- 6) Write MyBootApplication.java
- 7) Write the Required Bean Configurations in JLCAppConfig.java for Exchange, Queues and Bindings between Exchange and Queues.
- 8) Now **hello-micro-service-1** is ready to send the Messages to RabbitMQ Server.



hello-micro-service-1: Files required

1. MyBootApplication.java	2. HelloController.java
3. HelloService.java	4. JLCAppConfig.java
5. application.properties	6. pom.xml

1. MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```

2. HelloController.java

```
package com.jlcindia.springboot;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class HelloController {

    @Autowired
    HelloService helloService;

    @GetMapping(value = "/myhello/{name}")
    public String getMessage(@PathVariable String name) {
        System.out.println("HelloController - getMessage()");

        String msg = "Hello " + name + " - Welcome to JLC !!!";

        helloService.sendMessage(msg);

        return msg;
    }
}
```



3. HelloService.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
public class HelloService {

    @Autowired
    RabbitTemplate rabbitTemplate;

    public void sendMessage(String helloMsg) {
        System.out.println("HelloService - sendMessage");

        rabbitTemplate.convertAndSend("myhello-exchange-1", "myhello.key.2",helloMsg);

    }
}
```

4. JLCAppConfig.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCAppConfig {

    @Bean(name = "myHelloExchange")
    Exchange createHelloExchange() {
        return ExchangeBuilder.directExchange("myhello-exchange-1").build();
    }
}
```



```
@Bean(name = "myHelloQueue1")
Queue createHelloQueue1() {
    return QueueBuilder.durable("myhello-queue-1").build();
}

@Bean(name = "myHelloQueue2")
Queue createHelloQueue2() {
    return QueueBuilder.durable("myhello-queue-2").build();
}

@Bean(name = "myHelloQueue3")
Queue createHelloQueue3() {
    return QueueBuilder.durable("myhello-queue-3").build();
}

@Bean
Binding helloBinding1(Queue myHelloQueue1, DirectExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue1)
        .to(myHelloExchange)
        .with("myhello.key.1");
}

@Bean
Binding helloBinding2(Queue myHelloQueue2, DirectExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue2)
        .to(myHelloExchange)
        .with("myhello.key.2");
}

@Bean
Binding helloBinding3(Queue myHelloQueue3, DirectExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue3)
        .to(myHelloExchange)
        .with("myhello.key.3");
}
}
```

5. application.properties

```
spring.application.name=hello-micro-service-1
server.port=5000
```

```
# RabbitMQ Properties
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```



6. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hello-micro-service-1</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hello-micro-service-1</name>
<description>This is my Hello MicroService-1</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```



B)Develop the Hai MicroService -1

- 9) Copy **hello-micro-service-1** from previous module as **hai-micro-service-1**
- 10) Update pom.xml as follows.
- 11) Update application.properties
- 12) Write HaiService.java
- 13) Write MyBootApplication.java
- 14) Write the Required Bean Configurations for Exchange, Queues and Bindings between Echange and Queues
- 15) Now **hai-micro-service-1** is ready to receive the Messages from RabbitMQ Server.

hai-micro-service-1: Files required

1. MyBootApplication.java	2. HelloService.java
3. JLCAppConfig.java	4. application.properties
5. pom.xml	

1. MyBootApplication.java

```
package com.jlccindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```

2. HelloService.java

```
package com.jlccindia.springboot;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



@Service

```
public class HaiService {  
  
    @RabbitListener(queues = "myhello-queue-1")  
    public void onMessageArrived1(String helloMsg) {  
        System.out.println("HaiService - onMessageArrived for Queue1");  
        System.out.println("-----");  
        System.out.println("Message : " + helloMsg);  
        System.out.println("-----");  
    }  
  
    @RabbitListener(queues = "myhello-queue-2")  
    public void onMessageArrived2(String helloMsg) {  
        System.out.println("HaiService - onMessageArrived for Queue2");  
        System.out.println("-----");  
        System.out.println("Message : " + helloMsg);  
        System.out.println("-----");  
    }  
  
    @RabbitListener(queues = "myhello-queue-3")  
    public void onMessageArrived3(String helloMsg) {  
        System.out.println("HaiService - onMessageArrived for Queue3");  
        System.out.println("-----");  
        System.out.println("Message : " + helloMsg);  
        System.out.println("-----");  
    }  
}
```

3. JLCAppConfig.java

```
package com.jlcindia.springboot;  
  
import org.springframework.amqp.core.Binding;  
import org.springframework.amqp.core.BindingBuilder;  
import org.springframework.amqp.core.DirectExchange;  
import org.springframework.amqp.core.Exchange;  
import org.springframework.amqp.core.ExchangeBuilder;  
import org.springframework.amqp.core.Queue;  
import org.springframework.amqp.core.QueueBuilder;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.context.annotation.Bean;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
  
@SpringBootApplication  
public class JLCAppConfig {
```



```
@Bean(name = "myHelloExchange")
Exchange createHelloExchange() {
    return ExchangeBuilder.directExchange("myhello-exchange-1").build();
}

@Bean(name = "myHelloQueue1")
Queue createHelloQueue1() {
    return QueueBuilder.durable("myhello-queue-1").build();
}

@Bean(name = "myHelloQueue2")
Queue createHelloQueue2() {
    return QueueBuilder.durable("myhello-queue-2").build();
}

@Bean(name = "myHelloQueue3")
Queue createHelloQueue3() {
    return QueueBuilder.durable("myhello-queue-3").build();
}

@Bean
Binding helloBinding1(Queue myHelloQueue1, DirectExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue1)
        .to(myHelloExchange)
        .with("myhello.key.1");
}

@Bean
Binding helloBinding2(Queue myHelloQueue2, DirectExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue2)
        .to(myHelloExchange)
        .with("myhello.key.2");
}

@Bean
Binding helloBinding3(Queue myHelloQueue3, DirectExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue3)
        .to(myHelloExchange)
        .with("myhello.key.3");
}
}
```

4. application.properties

```
spring.application.name=hai-micro-service-1
server.port=9000
```

```
# RabbitMQ Properties
spring.rabbitmq.host=localhost
```



```
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hai-micro-service-1</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hai-micro-service-1</name>
<description>This is my Hai MicroService-1</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

C) Execution Steps

- 16) Start **hello-micro-service-1**
- 17) Start **hai-micro-service-1**
- 18) Open the browser and send the request
http://localhost:5000/myhello/Srinivas



4. FanoutExchange -Example

A)Develop the Hello MicroService-2

- 1) Copy **hello-micro-service-1** from previous module as **hello-micro-service-2**
- 2) Update pom.xml as follows.
- 3) Update application.properties
- 4) Update HelloService.java
- 5) Update JLCAppConfig.java.
- 6) Now **hello-micro-service-2** is ready to send the Messages to RabbitMQ Server.

hello-micro-service-2: Files required

1. MyBootApplication.java	Same as hello-micro-service-1
2. HelloController.java	Same as hello-micro-service-1
3. HelloService.java	Updated in Same as hello-micro-service-2
4. JLCAppConfig.java	Updated in Same as hello-micro-service-2
5. application.properties	Updated in Same as hello-micro-service-2
6. pom.xml	Updated in Same as hello-micro-service-2

3. HelloService.java

```
package com.jlccindia.springboot;
```

```
import org.springframework.amqp.rabbit.core.RabbitTemplate;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
*/
```

```
@Service
```

```
public class HelloService {
```

```
@Autowired
```

```
RabbitTemplate rabbitTemplate;
```

```
public void sendMessage(String helloMsg) {  
    System.out.println("HelloService - sendMessage");
```

```
    rabbitTemplate.convertAndSend("myhello-exchange-2", "myhello.key.2",helloMsg);
```

```
}
```

```
}
```



4. JLCAppConfig.java

```
package com.jlccindia.springboot;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.FanoutExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCAppConfig {

    @Bean(name = "myHelloExchange")
    Exchange createHelloExchange() {
        return ExchangeBuilder.fanoutExchange("myhello-exchange-2").build();
    }

    @Bean(name = "myHelloQueue1")
    Queue createHelloQueue1() {
        return QueueBuilder.durable("myhello-queue-1").build();
    }

    @Bean(name = "myHelloQueue2")
    Queue createHelloQueue2() {
        return QueueBuilder.durable("myhello-queue-2").build();
    }

    @Bean(name = "myHelloQueue3")
    Queue createHelloQueue3() {
        return QueueBuilder.durable("myhello-queue-3").build();
    }

    @Bean
    Binding helloBinding1(Queue myHelloQueue1, FanoutExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue1)
            .to(myHelloExchange);
    }
}
```



@Bean

```
Binding helloBinding2(Queue myHelloQueue2, FanoutExchange myHelloExchange) {  
    return BindingBuilder.bind(myHelloQueue2)  
        .to(myHelloExchange);  
}
```

@Bean

```
Binding helloBinding3(Queue myHelloQueue3, FanoutExchange myHelloExchange) {  
    return BindingBuilder.bind(myHelloQueue3)  
        .to(myHelloExchange);  
}  
  
}
```

5. application.properties

```
spring.application.name=hello-micro-service-2  
server.port=5000
```

RabbitMQ Properties

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

6. pom.xml

```
<project ...>
```

```
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.springboot</groupId>  
<artifactId>hello-micro-service-2</artifactId>  
<version>1.0</version>  
<packaging>jar</packaging>  
<name>hello-micro-service-2</name>  
<description>This is my Hello MicroService-2</description>
```

```
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.5.3</version>  
</parent>
```

```
<dependencies>
```

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

</plugins>
</build>
</project>
```

B)Develop the Hai MicroService -2

- 7) Copy **hai-micro-service-1** from previous module as **hai-micro-service-2**
- 8) Update pom.xml as follows.
- 9) Update application.properties
- 10) Update HaiService.java
- 11) Update JLCAppConfig.java
- 12) Now **hai-micro-service-2** is ready to receive the Messages from RabbitMQ Server.

hai-micro-service-2: Files required

1. MyBootApplication.java	Same as hai-micro-service-1
2. HaiService.java	Same as hai-micro-service-1
3. JLCAppConfig.java	Updated in hai-micro-service-2
4. application.properties	Updated in hai-micro-service-2
5. pom.xml	Updated in hai-micro-service-2

3. JLCAppConfig.java

```
package com.jlcindia.springboot;
```

```
import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.FanoutExchange;
import org.springframework.amqp.core.Queue;
```



```
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCAppConfig {

    @Bean(name = "myHelloExchange")
    Exchange createHelloExchange() {
        return ExchangeBuilder.fanoutExchange("myhello-exchange-2").build();
    }

    @Bean(name = "myHelloQueue1")
    Queue createHelloQueue1() {
        return QueueBuilder.durable("myhello-queue-1").build();
    }

    @Bean(name = "myHelloQueue2")
    Queue createHelloQueue2() {
        return QueueBuilder.durable("myhello-queue-2").build();
    }

    @Bean(name = "myHelloQueue3")
    Queue createHelloQueue3() {
        return QueueBuilder.durable("myhello-queue-3").build();
    }

    @Bean
    Binding helloBinding1(Queue myHelloQueue1, FanoutExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue1)
            .to(myHelloExchange);
    }

    @Bean
    Binding helloBinding2(Queue myHelloQueue2, FanoutExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue2)
            .to(myHelloExchange);
    }

    @Bean
    Binding helloBinding3(Queue myHelloQueue3, FanoutExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue3)
            .to(myHelloExchange);
    }
}
```



4. application.properties

```
spring.application.name=hai-micro-service-2  
server.port=9000
```

```
# RabbitMQ Properties  
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

5. pom.xml

```
<project ...>  
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.springboot</groupId>  
<artifactId>hai-micro-service-2</artifactId>  
<version>1.0</version>  
<packaging>jar</packaging>  
<name>hai-micro-service-2</name>  
<description>This is my Hai MicroService-2</description>  
  
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.5.3</version>  
</parent>  
  
<dependencies>  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>  
  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-amqp</artifactId>  
</dependency>  
</dependencies>  
  
<build>  
<plugins>  
<plugin>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-maven-plugin</artifactId>  
</plugin>  
</plugins>  
</build>  
</project>
```



C) Execution Steps

- 13) Start **hello-micro-service-2**
- 14) Start **hai-micro-service-2**
- 15) Open the browser and send the request
http://localhost:5000/myhello/Srinivas

5. TopicExchange –Example

A)Develop the Hello MicroService-3

- 1) Copy **hello-micro-service-2** from previous module as **hello-micro-service-3**
- 2) Update pom.xml as follows.
- 3) Update application.properties
- 4) Update HelloService.java
- 5) Update JLCAppConfig.java.
- 6) Now **hello-micro-service-3** is ready to send the Messages to RabbitMQ Server.

hello-micro-service-3: Files required

1. MyBootApplication.java	Same as hello-micro-service-2
2. HelloController.java	Same as hello-micro-service-2
3. HelloService.java	Updated in Same as hello-micro-service-3
4. JLCAppConfig.java	Updated in Same as hello-micro-service-3
5. application.properties	Updated in Same as hello-micro-service-3
6. pom.xml	Updated in Same as hello-micro-service-3

3. HelloService.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
public class HelloService {
```



@Autowired

RabbitTemplate rabbitTemplate;

```
public void sendMessage(String helloMsg) {  
    System.out.println("HelloService - sendMessage");  
  
    //rabbitTemplate.convertAndSend("myhello-exchange-3", "myhello",helloMsg);  
    //rabbitTemplate.convertAndSend("myhello-exchange-3", "myhello.key1",helloMsg);  
    rabbitTemplate.convertAndSend("myhello-exchange-3", "myhello.key.1",helloMsg);  
}  
}
```

4. JLCAppConfig.java

```
package com.jlcindia.springboot;  
  
import org.springframework.amqp.core.Binding;  
import org.springframework.amqp.core.BindingBuilder;  
import org.springframework.amqp.core.Exchange;  
import org.springframework.amqp.core.ExchangeBuilder;  
import org.springframework.amqp.core.Queue;  
import org.springframework.amqp.core.QueueBuilder;  
import org.springframework.amqp.core.TopicExchange;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.context.annotation.Bean;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
@SpringBootApplication  
public class JLCAppConfig {  
  
    @Bean(name = "myHelloExchange")  
    Exchange createHelloExchange() {  
        return ExchangeBuilder.topicExchange("myhello-exchange-3").build();  
    }  
  
    @Bean(name = "myHelloQueue1")  
    Queue createHelloQueue1() {  
        return QueueBuilder.durable("myhello-queue-1").build();  
    }  
  
    @Bean(name = "myHelloQueue2")  
    Queue createHelloQueue2() {  
        return QueueBuilder.durable("myhello-queue-2").build();  
    }  
}
```



```
@Bean(name = "myHelloQueue3")
Queue createHelloQueue3() {
    return QueueBuilder.durable("myhello-queue-3").build();
}

@Bean
Binding helloBinding1(Queue myHelloQueue1, TopicExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue1)
        .to(myHelloExchange)
        .with("myhello.*");
}

@Bean
Binding helloBinding2(Queue myHelloQueue2, TopicExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue2)
        .to(myHelloExchange)
        .with("myhello.#");
}

@Bean
Binding helloBinding3(Queue myHelloQueue3, TopicExchange myHelloExchange) {
    return BindingBuilder.bind(myHelloQueue3)
        .to(myHelloExchange)
        .with("myhello.key.*");
}
}
```

5. application.properties

```
spring.application.name=hello-micro-service-3
server.port=5000
```

```
# RabbitMQ Properties
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

6. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hello-micro-service-3</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hello-micro-service-3</name>
```



```
<description>This is my Hello MicroService-3</description>
```

```
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.5.3</version>  
</parent>
```

```
<dependencies>  
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-web</artifactId>  
</dependency>
```

```
<dependency>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-amqp</artifactId>  
</dependency>  
</dependencies>
```

```
<build>  
<plugins>  
<plugin>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-maven-plugin</artifactId>  
</plugin>  
</plugins>  
</build>  
</project>
```

B)Develop the Hai MicroService -3

- 7) Copy **hai-micro-service-2** from previous module as **hai-micro-service-3**
- 8) Update pom.xml as follows.
- 9) Update application.properties
- 10) Update HaiService.java
- 11) Update JLCAppConfig.java
- 12) Now **hai-micro-service-3** is ready to receive the Messages from RabbitMQ Server.

hai-micro-service-3: Files required

1. MyBootApplication.java	Same as hai-micro-service-2
2. HaiService.java	Same as hai-micro-service-2
3. JLCAppConfig.java	Updated in hai-micro-service-3
4. application.properties	Updated in hai-micro-service-3
5. pom.xml	Updated in hai-micro-service-3



3. JLCAppConfig.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCAppConfig {

    @Bean(name = "myHelloExchange")
    Exchange createHelloExchange() {
        return ExchangeBuilder.topicExchange("myhello-exchange-3").build();
    }

    @Bean(name = "myHelloQueue1")
    Queue createHelloQueue1() {
        return QueueBuilder.durable("myhello-queue-1").build();
    }

    @Bean(name = "myHelloQueue2")
    Queue createHelloQueue2() {
        return QueueBuilder.durable("myhello-queue-2").build();
    }

    @Bean(name = "myHelloQueue3")
    Queue createHelloQueue3() {
        return QueueBuilder.durable("myhello-queue-3").build();
    }

    @Bean
    Binding helloBinding1(Queue myHelloQueue1, TopicExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue1)
            .to(myHelloExchange)
            .with("myhello.*");
    }
}
```



```
@Bean
Binding helloBinding2(Queue myHelloQueue2, TopicExchange myHelloExchange) {
return BindingBuilder.bind(myHelloQueue2)
.to(myHelloExchange)
.with("myhello.#");
}

@Bean
Binding helloBinding3(Queue myHelloQueue3, TopicExchange myHelloExchange) {
return BindingBuilder.bind(myHelloQueue3)
.to(myHelloExchange)
.with("myhello.key.*");
}
}
```

4. application.properties

```
spring.application.name=hai-micro-service-3
server.port=9000

# RabbitMQ Properties
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hai-micro-service-3</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hai-micro-service-3</name>
<description>This is my Hai MicroService-3</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

C) Execution Steps

13) Start **hello-micro-service-3**

14) Start **hai-micro-service-3**

15) Open the browser and send the request

http://localhost:5000/myhello/Srinivas

6. HeadersExchange –Example

A)Develop the Hello MicroService-4

- 1) Copy **hello-micro-service-3** from previous module as **hello-micro-service-4**
- 2) Update pom.xml as follows.
- 3) Update application.properties
- 4) Update HelloService.java
- 5) Update JLCAppConfig.java.
- 6) Now **hello-micro-service-4** is ready to send the Messages to RabbitMQ Server.

hello-micro-service-4: Files required

1. MyBootApplication.java	Same as hello-micro-service-3
2. HelloController.java	Same as hello-micro-service-3
3. HelloService.java	Updated in Same as hello-micro-service-4
4. JLCAppConfig.java	Updated in Same as hello-micro-service-4
5. application.properties	Updated in Same as hello-micro-service-4
6. pom.xml	Updated in Same as hello-micro-service-4



3. HelloService.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.core.Message;
import org.springframework.amqp.core.MessageProperties;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.amqp.support.converter.SimpleMessageConverter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
public class HelloService {

    @Autowired
    private RabbitTemplate rabbitTemplate;

    public void sendMessage(String helloMsg) {
        System.out.println("HelloService - sendMessage()");

        MessageProperties messageProps = new MessageProperties();
        messageProps.setHeader("myheader", "JLC2");

        MessageConverter messageConverter = new SimpleMessageConverter();
        Message message = messageConverter.toMessage(helloMsg, messageProps);

        rabbitTemplate.convertAndSend("myhello.exchange-4","", message);
    }
}
```

4. JLCAppConfig.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.HeadersExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
```



```
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCAppConfig {

    @Bean(name="myHelloExchange")
    Exchange createHelloExchange() {
        return ExchangeBuilder.headersExchange("myhello-exchange-4").build();
    }

    @Bean(name="myHelloQueue1")
    Queue createHelloQueue1() {
        return QueueBuilder.durable("myhello-queue-1").build();
    }

    @Bean(name="myHelloQueue2")
    Queue createHelloQueue2() {
        return QueueBuilder.durable("myhello-queue-2").build();
    }

    @Bean(name="myHelloQueue3")
    Queue createHelloQueue3() {
        return QueueBuilder.durable("myhello-queue-3").build();
    }

    @Bean
    Binding helloBinding1(Queue myHelloQueue1, HeadersExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue1)
            .to(myHelloExchange).where("myheader").matches("JLC1");
    }

    @Bean
    Binding helloBinding2(Queue myHelloQueue2, HeadersExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue2)
            .to(myHelloExchange).where("myheader").matches("JLC2");
    }

    @Bean
    Binding helloBinding3(Queue myHelloQueue3, HeadersExchange myHelloExchange) {
        return BindingBuilder.bind(myHelloQueue3)
            .to(myHelloExchange).where("myheader").matches("JLC3");
    }

}
```



5. application.properties

```
spring.application.name=hello-micro-service-4  
server.port=5000
```

RabbitMQ Properties

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

6. pom.xml

```
<project ...>  
  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.jlcindia.springboot</groupId>  
  <artifactId>hello-micro-service-4</artifactId>  
  <version>1.0</version>  
  <packaging>jar</packaging>  
  <name>hello-micro-service-4</name>  
  <description>This is my Hello MicroService-4</description>  
  
  <parent>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-parent</artifactId>  
    <version>2.5.3</version>  
  </parent>  
  
  <dependencies>  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-web</artifactId>  
    </dependency>  
  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-amqp</artifactId>  
    </dependency>  
  </dependencies>  
  
  <build>  
    <plugins>  
      <plugin>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-maven-plugin</artifactId>  
      </plugin>  
    </plugins>  
  </build>  
</project>
```



B)Develop the Hai MicroService -4

- 7) Copy **hai-micro-service-3** from previous module as **hai-micro-service-4**
- 8) Update pom.xml as follows.
- 9) Update application.properties
- 10) Update HaiService.java
- 11) Update JLCAppConfig.java
- 12) Now **hai-micro-service-4** is ready to receive the Messages from RabbitMQ Server.

hai-micro-service-4: Files required

1. MyBootApplication.java	Same as hai-micro-service-3
2. HaiService.java	Same as hai-micro-service-3
3. JLCAppConfig.java	Updated in hai-micro-service-4
4. application.properties	Updated in hai-micro-service-4
5. pom.xml	Updated in hai-micro-service-4

3. JLCAppConfig.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.HeadersExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCAppConfig {

    @Bean(name="myHelloExchange")
    Exchange createHelloExchange() {
        return ExchangeBuilder.headersExchange("myhello-exchange-4").build();
    }
}
```



```
@Bean(name="myHelloQueue1")
Queue createHelloQueue1() {
return QueueBuilder.durable("myhello-queue-1").build();
}

@Bean(name="myHelloQueue2")
Queue createHelloQueue2() {
return QueueBuilder.durable("myhello-queue-2").build();
}

@Bean(name="myHelloQueue3")
Queue createHelloQueue3() {
return QueueBuilder.durable("myhello-queue-3").build();
}

@Bean
Binding helloBinding1(Queue myHelloQueue1, HeadersExchange myHelloExchange) {
return BindingBuilder.bind(myHelloQueue1)
.to(myHelloExchange).where("myheader").matches("JLC1");
}

@Bean
Binding helloBinding2(Queue myHelloQueue2, HeadersExchange myHelloExchange) {
return BindingBuilder.bind(myHelloQueue2)
.to(myHelloExchange).where("myheader").matches("JLC2");
}

@Bean
Binding helloBinding3(Queue myHelloQueue3, HeadersExchange myHelloExchange) {
return BindingBuilder.bind(myHelloQueue3)
.to(myHelloExchange).where("myheader").matches("JLC3");
}
}
```

4. application.properties

```
spring.application.name=hai-micro-service-4
server.port=9000

# RabbitMQ Properties
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```



5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>hai-micro-service-4</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>hai-micro-service-4</name>
<description>This is my Hai MicroService-4</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

C) Execution Steps

13) Start **hello-micro-service-4**

14) Start **hai-micro-service-4**

15) Open the browser and send the request

<http://localhost:5000/myhello/Srinivas>



7. Sending Order Message with Fanout Exchnage

jlC-book-store: Files required

1. MyBootApplication.java	2. Order.java
3. BookStoreController.java	4. BookStoreService.java
5. JLCAppConfig.java	6. application.properties
7. pom.xml	

1. MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```

2. Order.java

```
package com.jlcindia.springboot;

import java.io.Serializable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Order implements Serializable{

    private Integer orderId;
    private String userId;
    private int totalQty;
    private double totalCost;

    //Constructors
    //Setters and Getters
    //toString()

}
```



3. BookStoreController.java

```
package com.jlcindia.springboot;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class BookStoreController {

    @Autowired
    BookStoreService bookStoreService;

    @GetMapping(value="/myorder/{orderId}/{userId}/{totalQty}/{totalCost}")
    public String placeOrder(@PathVariable Integer orderId, @PathVariable String userId,
        @PathVariable int totalQty,@PathVariable double totalCost) {

        System.out.println("BookStoreController - placeOrder()");

        Order order=new Order(orderId, userId, totalQty, totalCost);
bookStoreService.placeOrder(order);

        return "Your Order is placed successfully !!!";
    }
}
```

4. BookStoreService.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
public class BookStoreService {

    @Autowired
    private RabbitTemplate rabbitTemplate;
```



```
public void placeOrder(Order order) {  
    System.out.println("BookStoreService - placeOrder()");  
  
    rabbitTemplate.convertAndSend("myorder.exchange","myorder.key", order);  
  
}  
}
```

5. JLCAppConfig.java

```
package com.jlcindia.springboot;  
  
import org.springframework.amqp.core.Binding;  
import org.springframework.amqp.core.BindingBuilder;  
import org.springframework.amqp.core.Exchange;  
import org.springframework.amqp.core.ExchangeBuilder;  
import org.springframework.amqp.core.FanoutExchange;  
import org.springframework.amqp.core.Queue;  
import org.springframework.amqp.core.QueueBuilder;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.context.annotation.Bean;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
  
@SpringBootApplication  
public class JLCAppConfig {  
  
    @Bean(name="myorderExchange")  
    Exchange createOrderExchange() {  
        return ExchangeBuilder.fanoutExchange("myorder.exchange").build();  
    }  
  
    @Bean(name="myorderQueue1")  
    Queue createOrderQueue1() {  
        return QueueBuilder.durable("myorder.queue-1").build();  
    }  
  
    @Bean(name="myorderQueue2")  
    Queue createOrderQueue2() {  
        return QueueBuilder.durable("myorder.queue-2").build();  
    }  
}
```



```
@Bean
Binding orderBinding1(Queue myorderQueue1, FanoutExchange myorderExchange) {
    return BindingBuilder.bind(myorderQueue1)
        .to(myorderExchange);
}

@Bean
Binding orderBinding2(Queue myorderQueue2, FanoutExchange myorderExchange) {
    return BindingBuilder.bind(myorderQueue2)
        .to(myorderExchange);
}
}
```

6. application.properties

```
spring.application.name=jlc-book-store
server.port=5000

# RabbitMQ Properties
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

7. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>jlc-book-store</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>jlc-book-store</name>
<description>This is jlc-book-store web application</description>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

jlc-order-manager: Files required

1. MyBootApplication.java	2. Order.java
3. HaiService.java	4. JLCAppConfig.java
5. application.properties	6. pom.xml

1. MyBootApplication.java

```
package com.jlcindia.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication {
    public static void main(String as[]) {
        SpringApplication.run(MyBootApplication.class, as);
    }
}
```

2. Order.java

```
package com.jlcindia.springboot;

import java.io.Serializable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class Order implements Serializable{
```



```
private Integer orderId;  
private String userId;  
private int totalQty;  
private double totalCost;
```

```
//Constructors  
//Setters and Getters  
//toString()  
  
}
```

3. OrderService.java

```
package com.jlcindia.springboot;
```

```
import org.springframework.amqp.rabbit.annotation.RabbitListener;  
import org.springframework.stereotype.Service;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
**/
```

```
@Service
```

```
public class OrderService {
```

```
    @RabbitListener(queues = "myorder.queue-1")
```

```
    public void onOrderPlaced1(Order myorder) {  
        System.out.println("OrderService --onOrderPlaced for Queue1");  
        System.out.println("----- ");  
        System.out.println("Order Info: "+myorder);  
        System.out.println("----- ");  
    }
```

```
    @RabbitListener(queues = "myorder.queue-2")
```

```
    public void onOrderPlaced2(Order myorder) {  
        System.out.println("OrderService --onOrderPlaced for Queue2");  
        System.out.println("----- ");  
        System.out.println("Order Info: "+myorder);  
        System.out.println("----- ");  
    }
```

```
}
```



4. JLCAppConfig.java

```
package com.jlcindia.springboot;

import org.springframework.amqp.core.Binding;
import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Exchange;
import org.springframework.amqp.core.ExchangeBuilder;
import org.springframework.amqp.core.FanoutExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
public class JLCAppConfig {

    @Bean(name="myorderExchange")
    Exchange createOrderExchange() {
        return ExchangeBuilder.fanoutExchange("myorder.exchange").build();
    }

    @Bean(name="myorderQueue1")
    Queue createOrderQueue1() {
        return QueueBuilder.durable("myorder.queue-1").build();
    }

    @Bean(name="myorderQueue2")
    Queue createOrderQueue2() {
        return QueueBuilder.durable("myorder.queue-2").build();
    }

    @Bean
    Binding orderBinding1(Queue myorderQueue1, FanoutExchange myorderExchange) {
        return BindingBuilder.bind(myorderQueue1)
            .to(myorderExchange);
    }

    @Bean
    Binding orderBinding2(Queue myorderQueue2, FanoutExchange myorderExchange) {
        return BindingBuilder.bind(myorderQueue2)
            .to(myorderExchange);
    }
}
```



6. application.properties

```
spring.application.name=jlc-order-manager  
server.port=9000
```

```
# RabbitMQ Properties  
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

7. pom.xml

```
<project ...>  
  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.jlcindia.springboot</groupId>  
  <artifactId>jlc-order-manager</artifactId>  
  <version>1.0</version>  
  <packaging>jar</packaging>  
  <name>jlc-order-manager</name>  
  <description>This is jlc-order-manager micro service</description>  
  
  <parent>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-parent</artifactId>  
    <version>2.5.3</version>  
  </parent>  
  
  <dependencies>  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-web</artifactId>  
    </dependency>  
  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-amqp</artifactId>  
    </dependency>  
  </dependencies>  
  
  <build>  
    <plugins>  
      <plugin>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-maven-plugin</artifactId>  
      </plugin>  
    </plugins>  
  </build>  
</project>
```



Execution Steps

- 1) Start **jlc-book-store**
- 2) Start **jlc-order-manager**
- 3) Open the browser and send the request

`http://localhost:5000/myorder/101/5001/5/500`

About Erlang:

Erlang is a programming language used to build massively scalable soft real-time systems with requirements on high availability.

The Erlang runtime system is designed for systems which can be used to build applications like:

- 1) Distributed
- 2) Fault-tolerant
- 3) Soft real-time
- 4) Highly available, non-stop applications
- 5) Hot swapping, where code can be changed without stopping a system.

Many companies are using Erlang in their production systems:

- Amazon uses Erlang to implement SimpleDB, providing database services as a part of the Amazon Elastic Compute Cloud (EC2).
- Yahoo! uses it in its social bookmarking service, Delicious, which has more than 5 million users and 150 million bookmarked URLs.
- Facebook uses Erlang to power the backend of its chat service, handling more than 100 million active users.
- WhatsApp uses Erlang to run messaging servers, achieving up to 2 million connected users per server.
- T-Mobile uses Erlang in its SMS and authentication systems.
- Motorola is using Erlang in call processing products in the public-safety industry.
- Ericsson uses Erlang in its support nodes, used in GPRS and 3G mobile networks worldwide.

Uninstalling RabbitMQ and Erlang OTP

If RabbitMQ and Erlang are not uninstalled completely, you may encounter problems attempting to install RabbitMQ and Erlang on the same machine.

To uninstall RabbitMQ and Erlang from the machine completely, do the following:

1. Open the Windows Control Panel.
2. Double-click Programs and Features.
3. In the list of currently installed programs, right-click RabbitMQ Server, and then click Uninstall.
4. In the list of currently installed programs, right-click Erlang OTP, and then click Uninstall.
5. Open the Windows Task Manager.
6. In the Task Manager, look for the process epmd.exe. If this process is still running, right-click it and then click End Process.
7. Delete all the installation directories for both RabbitMQ and Erlang.
8. Delete the file C:\Windows\.erlang.cookie (if present).
9. Go to the User folder: C:\Users\[username], and then delete the file .erlang.cookie.
10. Also in the User folder, go to AppData\Roaming\RabbitMQ.
11. Backup the Log folder to a secure location, and then delete the folder.